CSA04 – Operating Systems Assignment 2 (CO2)

Performance Analysis of OS Concurrency, Scheduling and Deadlock Avoidance

Student: [YOUR NAME]

Register Number: [YOUR REGISTER NUMBER]

GitHub Repository: CSA04--Assignment2

Course: CSA04 Operating Systems | Total: 50 marks

Execution evidence note: code and reference outputs are included. Replace machine-dependent timings and generated/reference evidence with your own Linux executions and terminal screenshots before submission.

Q1 – Processes and Process States (6)

A process moves through New, Ready, Running, Waiting and Terminated. A running process blocks into Waiting for I/O and returns to Ready when the event completes. The PCB stores PID/PPID, state, program counter, registers, scheduling information, memory mapping, open files, credentials and accounting information. Task01/process_demo.c demonstrates fork(), ps, PID/PPID and waitpid(). Inspect /proc//status while the child sleeps.

For 2,000 examination sessions, process-per-student provides strong isolation but duplicates address-space and kernel bookkeeping, increases memory use and can increase context-switch/TLB/cache costs. One thread per student is lighter and shares memory, but a thread crash can affect the whole server. A bounded worker pool with asynchronous I/O is recommended: cap CPU workers, queue work, and isolate untrusted proctoring/media decoders in separate processes. Complexity: O(N) session bookkeeping and O(N) descriptors; fixed worker resource count O(W).

Q2 – CPU Scheduling (8)

The supplied workload is P1(0,8), P2(1,4), P3(2,9), P4(3,5). Task02/scheduling.py computes FCFS, non-preemptive SJF, SRTF, priority, and RR Gantt segments, TAT, WT and RT. Task02/round_robin.c completes the Round Robin TODO. Since the assignment does not supply priorities, the script states an explicit assumption: P1=3, P2=1, P3=4, P4=2 (smaller is higher). Do not omit this assumption.

FCFS is simple but causes convoying; SJF improves average waiting if burst estimates are reliable but risks starvation; SRTF improves short-job response with extra preemption; priority needs ageing; RR balances fairness and response but quantum controls context-switch overhead. Recommend MLFQ for a mixed thumbnail/film service: short initial quantum, demote CPU-heavy jobs to longer quanta, age waiting jobs, and reserve service for long jobs. RR queue operations are O(1) each; list-scanning SJF/SRTF/priority are O(n) per dispatch (heap O(log n)).

Q3 – Threads (6)

Task03/camera_threads.c compares serial processing, one thread per camera (64), and pools of 4, 8 and 16. Threads share code, heap, globals and descriptors; each has registers, stack, program counter and thread-local storage. User-level threads are runtime-scheduled; kernel threads are OS-scheduled. Mapping models are many-to-one, one-to-one and many-to-many. Use mutexes/atomics for shared counters; unsynchronised increments are data races. Speed-up is serial time divided by parallel time. It plateaus at core count, memory bandwidth, cache contention, serial fractions and thread-management overhead. A process-based alternative isolates faults but consumes more memory and needs IPC to combine results. Worker-pool compute is O(NF), scheduling O(N), worker resources O(W).

Q4 – Inter-Process Communication (6)

Task04/pipe_ipc.c completes the pipe TODO and reads until read() returns 0. The parent must close its write end; otherwise the child does not receive EOF and can block indefinitely. The sample should report 6 readings, average 35.5 and maximum 41. Task04/shm_ipc.c uses POSIX shared memory plus a process-shared semaphore. Pipe transfer uses kernel buffering/copying and simple stream semantics; shared memory can avoid per-message kernel copies but needs explicit synchronisation. Benchmark 1, 10 and 100 MiB with repeated trials on your Linux machine; the included correctness demo is not a substitute for measured throughput. Both data paths are O(n) in bytes.

Q5 – Synchronisation (11)

Task05/bounded_buffer.c uses empty and full counting semaphores plus a mutex. The buffer indexes and consumption accounting are protected. Run 1 producer/1 consumer and 4 producers/4 consumers; run bash run_100.sh to repeat the test 100 times. Expected invariant: each of the 20 items is consumed once and sum=210. Peterson's algorithm satisfies mutual exclusion, progress and bounded waiting for two processes under sequential consistency; modern reordering requires atomics/fences. Readers-writers uses a reader count and resource lock; dining philosophers can avoid circular wait using a global resource order or waiter. A monitor encapsulates state with condition variables. Recommend per-train locks for booking as a first scalable step; global lock serialises unrelated trains, per-coach locking adds complexity and lock-order risks. Atomic fetch-add suits simple counters; spin locks waste CPU for long critical sections. Each buffer operation is O(1); storage O(buffer size).

Q6 – Deadlock and Banker (13)

The four necessary conditions are mutual exclusion, hold-and-wait, no preemption and circular wait. The sample resource-allocation cycle is Job A holding GPU/requesting licence, Job B holding licence/requesting storage, Job C holding storage/requesting GPU. Prevention breaks a condition; avoidance uses Banker and requires maximum claims; detection/recovery permits allocations and later detects/aborts; ostrich ignores rare deadlocks but is risky for shared lab equipment.

Task06/banker.c implements the safety and request algorithms. Need=Max−Allocation. A process can finish if Need≤Work, after which its allocation is returned to Work. For requests, validate against Need and Available, tentatively allocate, run safety, then grant or roll back. The supplied expected output is in the assignment. banker_bench.c measures 10, 100 and 1,000 processes with three resource types. Worst-case safety scan is $O(mn^2)$, auxiliary space O(n+m), input matrices O(nm). Recommend admission control for jobs with reliable maximum resource claims, resource ordering where possible, and detection/recovery as a fallback for uncertain external failures.

Final verification checklist

- ■ Replace student placeholders and add the actual GitHub URL.
- ■ Run every C program on Linux; record distro/kernel and `gcc --version`.
- ■ Capture genuine screenshots from your terminal for every task.
- ■ Replace reference timing files with your own repeated measurements.
- ■ Run Q5 100 times; retain evidence of all successful sums.
- ■ Run Q4 payload benchmark at 1, 10 and 100 MiB (pipe and shared memory).
- ■ Commit progressively, at least one commit per task, not one bulk commit.
- ■ Share the repository with course faculty.